CS 577 Deep Learning
Homework 3: Attention and Transformer Forward

Your Name (Hawk username: `yourname`)

Fall 2026

Part 0: Setup and Axes

All experiments were executed on a CPU with a fixed random seed of `577`. The notebook environment used the following versions:

- Python: `3.11.18`
- NumPy: `2.4.6`
- PyTorch: `2.15.0`
- Matplotlib: `3.11.2`
- Device: `CPU`

All 11 public tests passed successfully. These tests covered stable masked softmax, causal masking, scaled dot-product attention, head splitting and merging, multi-head self-attention and cross-attention, positional encodings, and Transformer block behavior.

The final test summary was:

```
Ran 11 tests in 0.047s

OK

Public-test return code: 0
```

No failures or errors were reported.

0.2 Axis and mask contract

The attention scores S, attention weights A, and output Y have the following shapes:

$$S : (B, H, T_q, T_k), \qquad A : (B, H, T_q, T_k), \qquad Y : (B, H, T_q, d_v).$$

Here, B is the batch size, H is the number of heads, T_q is the number of query tokens, T_k is the number of key tokens, d_k is the query/key feature size, and d_v is the value feature size.

We apply softmax along the last axis (T_k) because each query compares itself with all keys. Softmax gives a probability to each allowed key, and these probabilities add up to 1. If all keys are blocked, the probabilities are all zero.

The key mask first has shape (B, T_k). We reshape it to $(B, 1, 1, T_k)$ because each batch may have a different mask, but the same mask is used for all heads and queries within that batch.

NumPy broadcasting works by matching dimensions from right to left. A dimension of size 1 can expand to match the corresponding dimension in another array.

This allows the mask with shape $(B, 1, 1, T_k)$ to work with attention scores of shape (B, H, T_q, T_k).

Part 1: Hand Calculation

The given query, key, and value matrices are:

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}.$$

All three matrices have shape $(3, 2)$, and $d_k = d_v = 2$.

First, we calculate the dot products between queries and keys:

$$QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

The shape of $QK^\top$ is $(3, 3)$.

Next, we divide each score by $\sqrt{d_k} = \sqrt{2}$:

$$S = \frac{QK^\top}{\sqrt{2}} = \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

The scaled score matrix S has shape $(3, 3)$.

We apply softmax along each row to calculate the attention weights:

$$A = \text{softmax}(S) = \begin{bmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

The attention weight matrix A has shape $(3, 3)$. Each row sums to one.

Softmax calculation for cat. The scaled scores for the cat row are:

$$s_{\text{cat}} = \left[0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right].$$

The softmax formula is:

$$a_j = \frac{e^{s_j}}{\sum_k e^{s_k}}.$$

The common denominator is:

$$e^0 + e^{1/\sqrt{2}} + e^{1/\sqrt{2}}.$$

Using this denominator, the attention weights are:

$$a_{\text{The}} = \frac{e^0}{e^0 + 2e^{1/\sqrt{2}}} \approx 0.1978,$$

$$a_{\text{cat}} = a_{\text{sleeps}} = \frac{e^{1/\sqrt{2}}}{e^0 + 2e^{1/\sqrt{2}}} \approx 0.4011.$$

The attention weights for cat are:

$$a_{\text{cat,row}} = [0.1978,\ 0.4011,\ 0.4011].$$

Attention output. The attention output is calculated by multiplying the attention weights by the value matrix:

$$Y = AV = \begin{bmatrix} 1.6044 & 0.7967 \\ 1.4011 & 1.2033 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

The output matrix Y has shape $(3, 2)$.

Weighted sum for cat. The value vectors are:

$$V_{\text{The}} = [1, 0], \qquad V_{\text{cat}} = [0, 2], \qquad V_{\text{sleeps}} = [3, 1].$$

We multiply each value vector by its attention weight and add the results:

$$Y_{\text{cat}} = 0.1978[1, 0] + 0.4011[0, 2] + 0.4011[3, 1].$$

Calculating each output component separately gives:

$$Y_{\text{cat}} = [0.1978(1) + 0.4011(0) + 0.4011(3),$$

$$0.1978(0) + 0.4011(2) + 0.4011(1)]\,.$$

The final output for cat is:

$$\boxed{Y_{\text{cat}} = [1.4011,\ 1.2033]}$$

1.2 Causal attention

In causal attention, each query can attend only to the current token and previous tokens. Future tokens are blocked.

In our example:

- The can attend only to The.
- cat can attend to The and cat.
- sleeps can attend to all three tokens.

We use the causal mask M, where 1 means allowed and 0 means blocked:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

The mask has shape $(3, 3)$.

After applying the mask and softmax, the attention weights are:

$$A_{\text{causal}} = \begin{bmatrix} 1.0000 & 0 & 0 \\ 0.3302 & 0.6698 & 0 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

The weight matrix has shape $(3, 3)$. Blocked keys have attention weights of exactly zero.

The new attention output is:

$$Y_{\text{causal}} = A_{\text{causal}} V = \begin{bmatrix} 1.0000 & 0 \\ 0.3302 & 1.3395 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

The output matrix has shape $(3, 2)$.

Weighted sum for cat. The cat query can attend only to The and cat. The sleeps token is blocked.

The attention weights for cat are:

$$a_{\text{cat}} = [0.3302,\ 0.6698,\ 0].$$

Using the value vectors, we calculate:

$$Y_{\text{cat}} = 0.3302[1, 0] + 0.6698[0, 2] + 0[3, 1].$$

Calculating the two output components separately:

$$\begin{aligned} Y_{\text{cat}} &= \big[0.3302(1) + 0.6698(0) + 0(3), \\ &\qquad 0.3302(0) + 0.6698(2) + 0(1)\big] \\ &\approx [0.3302,\ 1.3396]. \end{aligned}$$

Why the last row does not change: The last token, sleeps, can attend to all three tokens because there are no future tokens after it. Therefore, its attention weights and output remain the same as in unmasked attention.

Why each output has two values: Each value vector has two features $d_v = 2$. Attention calculates a weighted sum of these vectors. The causal mask changes which tokens are used, but it does not change the number of output features. That's why each output vector still has two values.

1.3 Scaling and stability

The dot product between a query and a key is:

$$s = q^\top k = \sum_{r=0}^{d_k-1} q_r k_r.$$

We assume that all coordinates are mutually independent, have zero mean, and unit variance:

$$E[q_r] = E[k_r] = 0, \qquad \mathrm{Var}(q_r) = \mathrm{Var}(k_r) = 1.$$

Using the variance formula and the zero mean:

$$\begin{aligned} \mathrm{Var}(q_r) = E[q_r^2] - (E[q_r])^2 = 1, \\ E[q_r^2] = 1. \end{aligned}$$

Similarly, $E[k_r^2] = 1$.

For each product, independence gives:

$$E[q_r k_r] = E[q_r]E[k_r] = 0.$$

$$E[q_r^2 k_r^2] = E[q_r^2]E[k_r^2] = 1.$$

Therefore,

$$\begin{aligned} \mathrm{Var}(q_r k_r) &= E[(q_r k_r)^2] - (E[q_r k_r])^2 \\ &= 1 - 0^2 \\ &= 1. \end{aligned}$$

Since all coordinates are mutually independent, each product uses a different independent pair. Therefore, the variances of the products add:

$$\begin{aligned}\mathrm{Var}(s) &= \mathrm{Var}\left(\sum_{r=0}^{d_k-1} q_r k_r\right) \\ &= \sum_{r=0}^{d_k-1} \mathrm{Var}(q_r k_r) \\ &= \sum_{r=0}^{d_k-1} 1 \\ &= \boxed{d_k}.\end{aligned}$$

After scaling by $\sqrt{d_k}$:

$$s_{\text{scaled}} = \frac{s}{\sqrt{d_k}}.$$

Using $\mathrm{Var}(cX) = c^2 \,\mathrm{Var}(X)$:

$$\begin{aligned}\mathrm{Var}(s_{\text{scaled}}) &= \mathrm{Var}\left(\frac{s}{\sqrt{d_k}}\right) \\ &= \frac{1}{d_k}\,\mathrm{Var}(s) \\ &= \frac{d_k}{d_k} \\ &= \boxed{1}.\end{aligned}$$

Why scaling is important: Without scaling, the standard deviation of the attention scores is $\sqrt{d_k}$. As d_k grows, the scores can become more spread out.

For two softmax weights, their ratio is:

$$\frac{a_i}{a_j} = e^{s_i - s_j}.$$

This means the ratio grows exponentially when the difference between scores increases. As a result, the largest score can receive almost all the attention weight, while the others receive values close to zero.

Dividing by $\sqrt{d_k}$ keeps the score variance controlled and reduces this problem.

Numerical stability: For numerical stability, we subtract the maximum score from each row before applying softmax:

$$m = \max_j s_j.$$

The softmax formula becomes:

$$\begin{aligned}\text{softmax}(s_i) &= \frac{e^{s_i}}{\sum_j e^{s_j}} \\ &= \frac{e^{s_i - m} e^m}{\sum_j e^{s_j - m} e^m} \\ &= \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}.\end{aligned}$$

The common factor e^m cancels, so subtracting the maximum does not change the softmax result.

After subtraction, the largest shifted score is zero, and its exponential is:

$$e^0 = 1.$$

All other shifted scores are less than or equal to zero, so their exponentials are at most one.

For a row with at least one allowed key, the denominator contains at least one term equal to one. Therefore, it is at least one, which avoids division by zero and denominator underflow.

For example, consider the scores:

$$[1000,\ 999,\ -1000].$$

Computing e^{1000} directly can cause overflow. After subtracting the maximum, we obtain:

$$[0,\ -1,\ -2000].$$

The exponentials are now:

$$[1,\ e^{-1},\ e^{-2000}].$$

This makes the calculation numerically stable without changing the softmax probabilities.

A fully masked row is handled separately by returning zeros, as required by the implementation.

Part 2: Stable Masked Attention

2.1 Numerical stability and fully blocked rows

TODO

2.2 Hand-case agreement

TODO

2.3 Padding and causal masks

TODO

Part 3: Multi-Head Attention

3.1 Shape trace, attention maps, and outputs

Stage	Meaning	Shape
X	input features	TODO
XW_Q, XW_K, XW_V	projected features	TODO
Q, K, V	after splitting heads	TODO
S, A	scores and weights per head	TODO
Head outputs	weighted values per head	TODO
Merged outputs	concatenated head features	TODO
Y_{MHA}	after output projection	TODO

TODO

3.2 Independent PyTorch comparison

Case	Maximum output error	Maximum weight error
Unmasked self-attention	TODO	TODO
Causal self-attention	TODO	TODO
Rectangular cross-attention	TODO	TODO

TODO

3.3 Head count and head-output ablation

TODO

Part 4: Transformer Block

4.1 Shape trace and reference checks

TODO

4.2 Positions and reordering

TODO

4.3 Future perturbation and causal visibility

TODO

Part 5: Architecture Comparison

5.1 Encoder–decoder and decoder-only generation

TODO

Assistance and Generative-AI Disclosure

TODO